

Fundamentals for Robotics and Automation

Prof. Gerardo Flores

January 2026

Robotics and Automation (CSCE 3345 - SENG 3340)

TAMU



Mathematical Foundations

1. Set of Real Numbers:

- $\mathbb{R}$: Denotes the set of real numbers.
- $\mathbb{R}^n$: Represents the vector space of n -tuples over $\mathbb{R}$.

2. Notations:

- **Scalars**: Represented by lowercase letters $a, b, c, x, y, \dots$ (e.g., $a \in \mathbb{R}$).
- **Vectors**: Represented as column vectors. For $x \in \mathbb{R}^n$:

$$x = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}, \quad x_i \in \mathbb{R}.$$

Alternatively:

$$x = [x_1, x_2, \dots, x_n]^T,$$

where T denotes the transpose.

- **Matrices**: Represented by uppercase letters $A, B, C, R, \dots$

Mathematical Foundations

3. Vector Norm:

- The length or norm of a vector $x \in \mathbb{R}^n$ is given by:

$$\|x\| = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}.$$

Python Example

```
import numpy as np

# Define the vector
X = np.array([-2, 3, -1])

# Compute the norm of the vector
norm_X = np.linalg.norm(X)

# Output the result
print(f"The norm of X is: {norm_X:.4f}")
```

- **f (formatted string literal)**: Enables dynamic insertion of variables and expressions within a string.
- **{variable}**: Indicates where the variable value will be placed in the output text.
- **:.4f**: Specifies that the number should be printed with four digits after the decimal point.

```
print(f"The norm of X is: {norm_X}")
```

MATRICES (Notation)

1. Definition:

- A real $m \times n$ matrix is represented as:

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,n} \\ a_{2,1} & a_{2,2} & \dots & a_{2,n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m,1} & a_{m,2} & \dots & a_{m,n} \end{bmatrix}, \quad A \in \mathbb{R}^{m \times n}.$$

- Where m is the number of rows and n is the number of columns.
- If $m = n$, the matrix is square.

Matrix Multiplication

- Two matrices A and B can be multiplied if the number of columns in A equals the number of rows in B .
- If A is $m \times n$ and B is $n \times p$, the resulting matrix C will be $m \times p$.
- The entry in row i and column j of C is computed as:

$$C[i, j] = \sum_{k=1}^n A[i, k] \cdot B[k, j]$$

Example:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$C = A \cdot B = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 7 & 1 \cdot 6 + 2 \cdot 8 \\ 3 \cdot 5 + 4 \cdot 7 & 3 \cdot 6 + 4 \cdot 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

```
import numpy as np

# Define matrices A and B
A = np.array([[1, 2],
              [3, 4]])
B = np.array([[5, 6],
              [7, 8]])

# Perform matrix multiplication
C = np.dot(A, B)

# Print the result
print("Matrix A:")
print(A)
print("\nMatrix B:")
print(B)
print("\nMatrix Multiplication Result (C = A * B):")
print(C)
```

Matrix-Vector Multiplication

- A matrix A can be multiplied by a vector $\mathbf{v}$ if the number of columns in A equals the number of entries in $\mathbf{v}$.
- If A is $m \times n$ and $\mathbf{v}$ is $n \times 1$, the resulting vector $\mathbf{u}$ will be $m \times 1$.
- The i -th entry of $\mathbf{u}$ is computed as:

$$u[i] = \sum_{k=1}^n A[i, k] \cdot v[k]$$

Example:

$$A = \begin{bmatrix} 2 & 1 \\ 0 & 3 \end{bmatrix}, \quad \mathbf{v} = \begin{bmatrix} 4 \\ 5 \end{bmatrix}$$
$$\mathbf{u} = A \cdot \mathbf{v} = \begin{bmatrix} 2 \cdot 4 + 1 \cdot 5 \\ 0 \cdot 4 + 3 \cdot 5 \end{bmatrix} = \begin{bmatrix} 13 \\ 15 \end{bmatrix}$$

```
import numpy as np

# Define matrix A and vector v
A = np.array([[2, 1],
              [0, 3]])
v = np.array([4, 5])

# Perform matrix-vector multiplication
u = np.dot(A, v)

# Print the result
print("Matrix A:")
print(A)
print("\nVector v:")
print(v)
print("\nMatrix-Vector Multiplication Result (u = A * v):")
print(u)
```

Scalar Product

1. Definition:

- The scalar product (dot product) of two vectors $x, y \in \mathbb{R}^n$ is defined as:

$$\langle x, y \rangle = x^T y = x_1 y_1 + x_2 y_2 + \cdots + x_n y_n.$$

2. Properties:

- Norm Relation:** The norm of a vector can be derived from the scalar product:

$$\|x\| = \sqrt{\langle x, x \rangle}.$$

- Commutativity:**

$$\langle x, y \rangle = \langle y, x \rangle.$$

3. Useful Inequalities:

- Cauchy-Schwarz:**

$$|\langle x, y \rangle| \leq \|x\| \|y\|.$$

- Triangle Inequality:**

$$\|x + y\| \leq \|x\| + \|y\|.$$

Scalar Product



vector_analysis.py

4. Geometric Interpretation (for vectors in $\mathbb{R}^3$):

- The scalar product can also be expressed as:

$$\langle x, y \rangle = \|x\| \|y\| \cos(\theta),$$

where θ is the angle between x and y .

```
import numpy as np

# Define two vectors
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

# Compute the dot (scalar) product
dot_product = np.dot(x, y)
print(f"Dot product ⟨x, y⟩: {dot_product}")

# Compute the norms of x and y
norm_x = np.linalg.norm(x)
norm_y = np.linalg.norm(y)
print(f"||x||: {norm_x:.4f}")
print(f"||y||: {norm_y:.4f}")

# Verify Cauchy-Schwarz inequality
cauchy_schwarz = abs(dot_product) <= norm_x * norm_y
print(f"Cauchy-Schwarz inequality holds: {cauchy_schwarz}")

# Verify Triangle inequality
triangle_inequality = np.linalg.norm(x + y) <= norm_x + norm_y
print(f"Triangle inequality holds: {triangle_inequality}")

# Geometric interpretation (cosine of the angle)
cos_theta = dot_product / (norm_x * norm_y)
theta = np.arccos(cos_theta) # Angle in radians
print(f"cos(θ): {cos_theta:.4f}")
print(f"Angle θ (in degrees): {np.degrees(theta):.2f}")
```

What does *linear* mean?

A function, system, or equation is **linear** if it satisfies the principle of superposition:

1. Additivity:

$$f(x_1 + x_2) = f(x_1) + f(x_2)$$

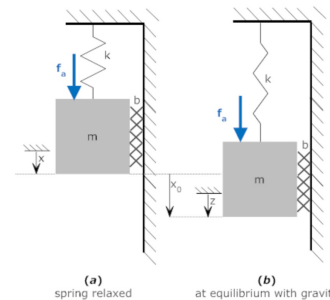
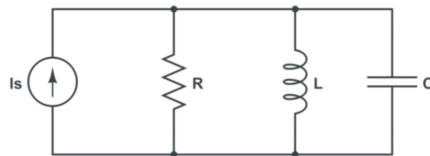
2. Homogeneity (scaling):

$$f(\alpha x) = \alpha f(x)$$

In words: if you double the input, the output doubles; if you add inputs, their effects add up. It behaves in a “proportional, predictable” way.

Linear examples:

- $f(x) = 3x$.
- Linear ODE: $\dot{x} = Ax$.
- Ideal R–L–C circuits.



$$5y'' - 2y' + 5y = 0$$

Goal: Show that

$$5y'' - 2y' + 5y = 0$$

is a **linear differential equation**.

Proof (operator method):

Define the operator $\mathcal{L}[y] = 5y'' - 2y' + 5y$.

Take y_1, y_2 twice-differentiable and $\alpha, \beta \in \mathbb{R}$. Then

$$\begin{aligned}\mathcal{L}[\alpha y_1 + \beta y_2] &= 5(\alpha y_1 + \beta y_2)'' - 2(\alpha y_1 + \beta y_2)' + 5(\alpha y_1 + \beta y_2) \\ &\stackrel{(2)}{=} 5(\alpha y_1'' + \beta y_2'') - 2(\alpha y_1' + \beta y_2') + 5(\alpha y_1 + \beta y_2) \\ &= \alpha (5y_1'' - 2y_1' + 5y_1) + \beta (5y_2'' - 2y_2' + 5y_2) \\ &= \alpha \mathcal{L}[y_1] + \beta \mathcal{L}[y_2].\end{aligned}$$

This holds because differentiation is linear. Hence $\mathcal{L}$ is a **linear operator**, so the equation $\mathcal{L}[y] = 0$ is a **linear homogeneous ODE**.

What does *nonlinear* mean?

A system is **nonlinear** if it breaks either rule above.

Nonlinear examples:

- $f(x) = x^2$.
- Motion equations with velocity-dependent friction.
- Mobile robot kinematics: $\dot{x} = v \cos \theta$, $\dot{y} = v \sin \theta$.
- Saturation, hysteresis, collisions, etc.

Example: Mobile robot kinematics (nonlinear)

Consider a differential-drive robot moving with linear velocity v and heading angle θ .

The kinematic model is:

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega$$

Why is it nonlinear?

Take the state vector $z = (x, y, \theta)$.

The dynamics can be written as:

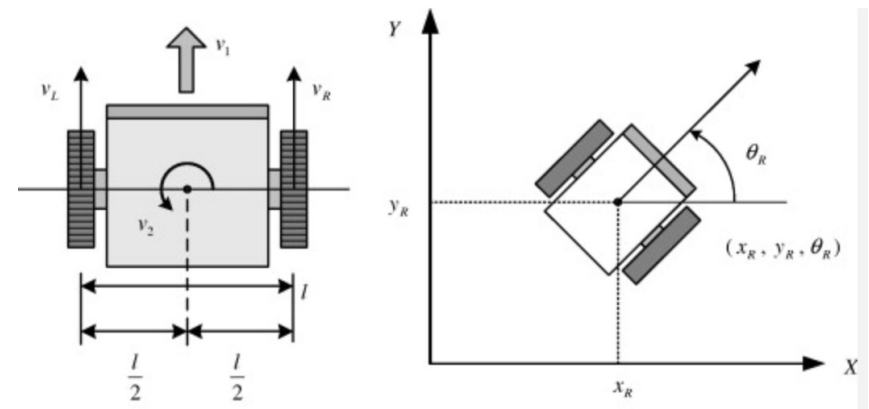
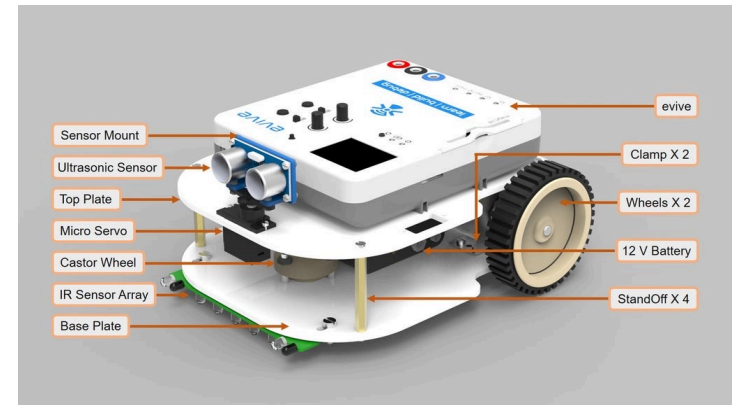
$$\dot{z} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix}.$$

- The terms $\cos \theta$ and $\sin \theta$ are **nonlinear functions** of the state θ .
- If you try the **superposition test**, it fails:

$$f(z_1 + z_2) \neq f(z_1) + f(z_2).$$

$$\text{(e.g. } \cos(\theta_1 + \theta_2) \neq \cos \theta_1 + \cos \theta_2 \text{)}.$$

Thus, this system is **nonlinear**.



Quiz: Linear or Nonlinear?

Q1. Is the equation

$$y'' + 3y' + 2y = 0$$

linear or nonlinear?

Q2. Is the function

$$f(x) = 7x$$

linear or nonlinear?

Q3. Is the equation

$$y'' + y^2 = 0$$

linear or nonlinear?

Q4. A mobile robot has the kinematics

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta.$$

Are these equations linear or nonlinear?

Q5. True or False:

"If you double the input of a linear system, the output also doubles."

Visual Studio Code installation



<https://code.visualstudio.com/docs/setup/mac> [choose your operating system]

<https://code.visualstudio.com/docs/python/python-tutorial> Python tutorial for visual studio code including virtual environment setup

Trace of a Matrix

The **trace** of a matrix A is defined as:

$$\text{tr}(A) = \sum_{i=1}^n A_{ii}$$

where A_{ii} represents the diagonal elements of the matrix.

Additionally, the trace can also be expressed as the sum of the eigenvalues λ_i of the matrix:

$$\text{Tr}(A) = \sum_{i=1}^n \lambda_i$$

Exercise

Find the trace of the matrix

Given the matrix A :

$$A = \begin{bmatrix} 5 & 3 & 5 \\ 4 & -1 & 2 \\ -3 & 8 & 7 \end{bmatrix}$$

The trace is defined as the sum of its diagonal elements:

$$\text{tr}(A) = 5 + (-1) + 7 = 11$$

```
import numpy as np
```

```
# Define the matrix
```

```
A = np.array([[5, 3, 5],  
              [4, -1, 2],  
              [-3, 8, 7]])
```

```
# Calculate the trace
```

```
trace_A = np.trace(A)
```

```
print("Trace of matrix A:", trace_A)
```

Answer:

$$\text{Trace}(A) = 11$$

Exercises

1. Find the dot product

Compute the dot product of two vectors:

$$a = \begin{bmatrix} -12 \\ 2 \\ -2 \end{bmatrix}, \quad b = \begin{bmatrix} -14 \\ 10 \\ -4 \end{bmatrix}$$

2. Verify the norm relation

Verify that the norm of a vector can be derived from the dot product:

$$\|a\| = \sqrt{\langle a, a \rangle}$$

For $a = \begin{bmatrix} -12 \\ 2 \\ -2 \end{bmatrix}$, calculate $\|a\|$.

```
import numpy as np
```

```
# Define vectors
```

```
a = np.array([-12, 2, -2])
```

```
b = np.array([-14, 10, -4])
```

```
# Calculate the dot product
```

```
dot_product = np.dot(a, b)
```

```
print("Dot Product:", dot_product)
```

Answer:

Dot Product = 160

```
# Norm of vector a
```

```
norm_a = np.sqrt(np.dot(a, a))
```

```
print("Norm of vector a:", norm_a)
```

Answer:

$\|a\| = 12.3288$

Exercises

Check the Cauchy-Schwarz inequality

Confirm that:

$$|\langle a, b \rangle| \leq \|a\| \cdot \|b\|$$

```
# Norm of vector b
norm_b = np.sqrt(np.dot(b, b))

# Cauchy-Schwarz inequality
left_side = abs(np.dot(a, b))
right_side = norm_a * norm_b
print("Cauchy-Schwarz inequality holds:", left_side <= right_side)
```

Answer:

The inequality holds because:

$$|\langle a, b \rangle| = 160, \quad \|a\| \cdot \|b\| = 169.651$$

Expressing Vectors as Linear Combinations of Unit Vectors

We define the standard unit vectors in $\mathbb{R}^3$ as follows:

$$\mathbf{i} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{j} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad \mathbf{k} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Using this notation, any vector $\mathbf{x}$ in $\mathbb{R}^3$, represented as:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix},$$

can also be written as:

$$\mathbf{x} = x_1\mathbf{i} + x_2\mathbf{j} + x_3\mathbf{k}.$$

```
import numpy as np

# Define the unit vectors
i = np.array([1, 0, 0])
j = np.array([0, 1, 0])
k = np.array([0, 0, 1])

# Define the vector x
x = np.array([2, -3, 4])

# Express x as a combination of i, j, k
x1, x2, x3 = x
linear_combination = x1 * i + x2 * j + x3 * k

# Print the result
print("Vector x:", x)
print("Linear combination: {}i + {}j + {}k".format(x1, x2, x3))
```

```
Vector x: [ 2 -3  4]
Linear combination: 2i + -3j + 4k
```

Vector Product or Cross Product

1. **Definition:** The vector product (cross product) of two vectors x and y in $\mathbb{R}^3$ is a vector c , defined as:

$$c = x \times y = \det \begin{bmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ x_1 & x_2 & x_3 \\ y_1 & y_2 & y_3 \end{bmatrix}$$

Expanding this determinant, the result is:

$$c = (x_2y_3 - x_3y_2)\mathbf{i} + (x_3y_1 - x_1y_3)\mathbf{j} + (x_1y_2 - x_2y_1)\mathbf{k}.$$

2. **Magnitude of the Cross Product:** The magnitude of the cross product $\|c\|$ is given by:

$$\|c\| = \|x\|\|y\| \sin(\theta),$$

where θ is the angle between the vectors x and y .

3. **Direction:** The direction of the vector c is determined by the **right-hand rule**.

- Curl the fingers of your right hand from vector x towards vector y , and your thumb will point in the direction of c .

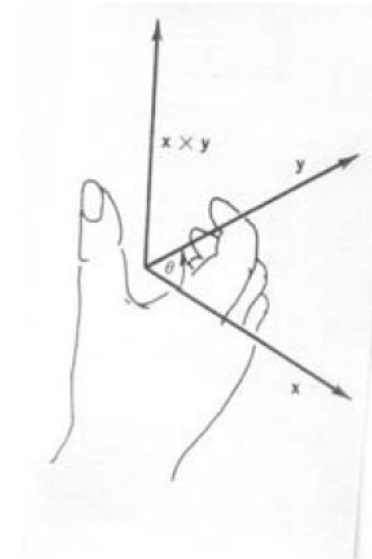
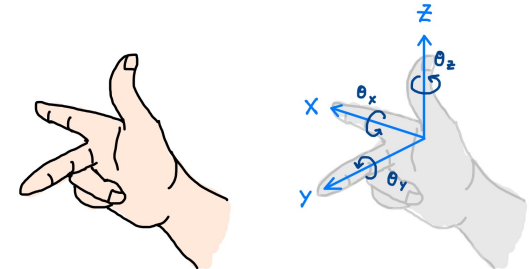


Fig. B.1 The right hand rule.



4. Properties of the Cross Product:

1. Anticommutativity:

$$\mathbf{x} \times \mathbf{y} = -\mathbf{y} \times \mathbf{x}$$

2. Distributivity:

$$\mathbf{x} \times (\mathbf{y} + \mathbf{z}) = \mathbf{x} \times \mathbf{y} + \mathbf{x} \times \mathbf{z}$$

3. Scalar Multiplication:

$$\alpha(\mathbf{x} \times \mathbf{y}) = (\alpha\mathbf{x}) \times \mathbf{y} = \mathbf{x} \times (\alpha\mathbf{y})$$

These properties highlight key algebraic behaviors of the cross product.

$$\mathbf{c} = (x_2y_3 - x_3y_2)\mathbf{i} + (x_3y_1 - x_1y_3)\mathbf{j} + (x_1y_2 - x_2y_1)\mathbf{k}.$$

Exercises

Given:

Two vectors $\mathbf{u} = 3\mathbf{i} - \mathbf{j} + \mathbf{k}$ and $\mathbf{v} = 2\mathbf{i} - 3\mathbf{j} + \mathbf{k}$.

Tasks:

1. Find the cross product $\mathbf{u} \times \mathbf{v}$ and verify that this vector is orthogonal to both $\mathbf{u}$ and $\mathbf{v}$.
2. Compute $\mathbf{v} \times \mathbf{u}$ and compare it with $\mathbf{u} \times \mathbf{v}$.

-
1. Computing the cross product $\mathbf{u} \times \mathbf{v}$:

$$\mathbf{u} \times \mathbf{v} = (2, -1, -7)$$

2. Verifying orthogonality:

- $\mathbf{u} \cdot (\mathbf{u} \times \mathbf{v}) = 0$
- $\mathbf{v} \cdot (\mathbf{u} \times \mathbf{v}) = 0$

This confirms that the resulting vector is orthogonal to both $\mathbf{u}$ and $\mathbf{v}$.

3. Computing the cross product $\mathbf{v} \times \mathbf{u}$:

$$\mathbf{v} \times \mathbf{u} = (-2, 1, 7)$$

We observe that $\mathbf{v} \times \mathbf{u} = -(\mathbf{u} \times \mathbf{v})$, which is consistent with the antisymmetric property of the cross product.



cross_product_verification.py

```

1  import numpy as np
2
3  # Define vectors u and v
4  u = np.array([3, -1, 1])
5  v = np.array([2, -3, 1])
6
7  # Compute the cross product u x v
8  cross_u_v = np.cross(u, v)
9
10 # Compute the cross product v x u
11 cross_v_u = np.cross(v, u)
12
13 # Verify orthogonality: dot product should be 0
14 # If u x v is orthogonal to both u and v, then their dot product should be zero
15 dot_u_cross = np.dot(u, cross_u_v)
16 dot_v_cross = np.dot(v, cross_u_v)
17
18 # Print results
19 print("Cross product u x v:", cross_u_v)
20 print("Cross product v x u:", cross_v_u)
21 print("Dot product u · (u x v):", dot_u_cross)
22 print("Dot product v · (u x v):", dot_v_cross)
23

```

Problem: Orientation and Forces in an Aerial Robotic Manipulator

A drone equipped with a robotic arm needs to lift an object by applying force in a specific direction. To control it, compute and verify the following:

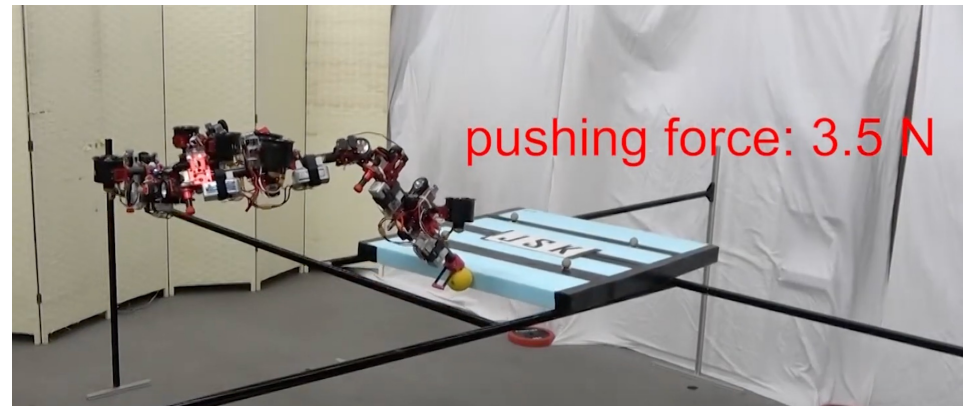
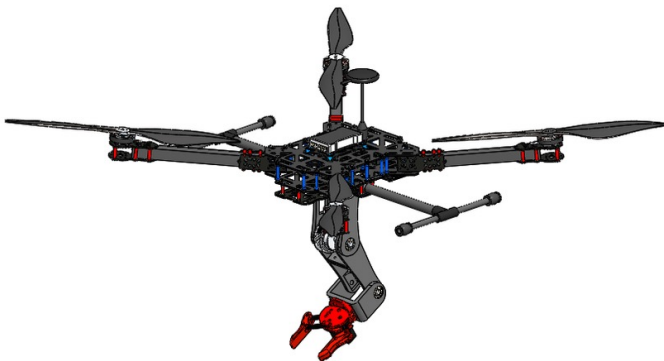
1. Force Direction (Normalization):

The arm applies a force in the direction of the vector

$$f = \begin{bmatrix} 2 \\ -3 \\ 4 \end{bmatrix}$$

Normalize this vector (using the **norm**) to obtain the unit direction of the force.

<https://youtube.com/clip/UgkxpkA3WfkCLHl-gc5NP6rkj70xCPIPS7Tw?si=maclIGK7rNUfr0VI>





1. Unit force direction

$$\|f\| = \sqrt{2^2 + (-3)^2 + 4^2} = \sqrt{29} \approx 5.3852$$

$$\hat{f} = \frac{f}{\|f\|} \approx \begin{bmatrix} 0.3714 \\ -0.5571 \\ 0.7428 \end{bmatrix}$$

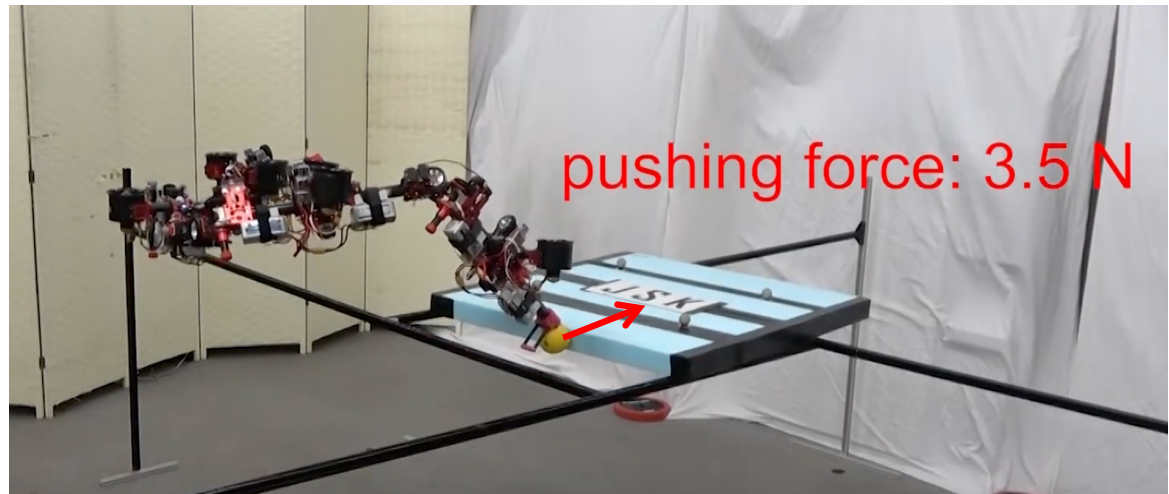


2. Work Performed (Dot Product):

If the drone displaces the object along the vector

$$d = \begin{bmatrix} -1 \\ 2 \\ 1 \end{bmatrix}$$

Compute the work performed by the arm using the **dot product** $W = f \cdot d$.





2) Work (dot product)

$$W = f \cdot d = 2(-1) + (-3)(2) + 4(1) = -4$$

Interpretation: The work is **negative**, meaning the force has a component opposite to the displacement.

When the **work is negative**, it means the **force opposes the displacement** of the object. In physics and robotics, this happens often:

- **Example 1:** Friction force when a robot slides along the floor. The displacement is forward, but the friction force points backward → negative work.



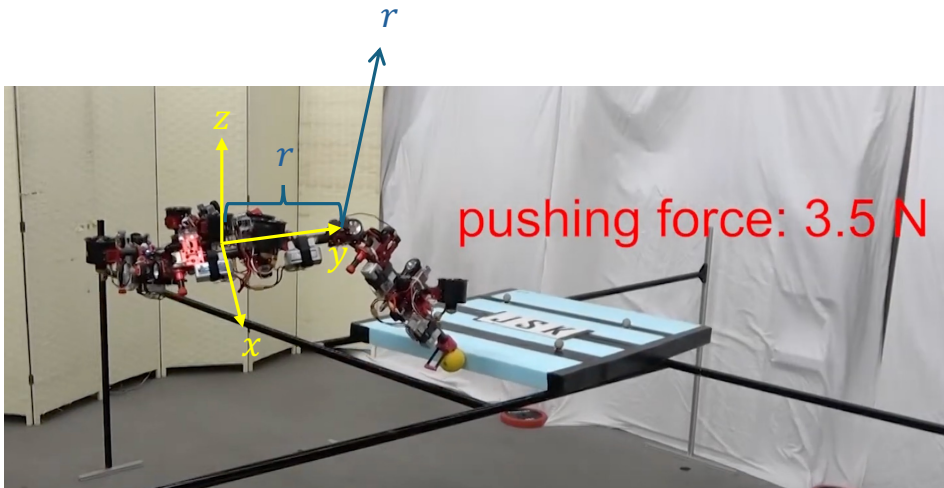
3. Torque (Cross Product):

The arm is attached to the drone at the point

$$r = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Compute the torque applied on the drone:

$$\tau = r \times f$$



3) Torque (cross product)

$$\tau = r \times f = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \times \begin{bmatrix} 2 \\ -3 \\ 4 \end{bmatrix} = \begin{bmatrix} 4 \\ 0 \\ -2 \end{bmatrix}$$

Direction given by the **right-hand rule**; magnitude:

$$\|\tau\| = \sqrt{4^2 + 0^2 + (-2)^2} = \sqrt{20} \approx 4.4721$$

4. System Stability (Trace of a Matrix):

The drone's control system is described by the gain matrix:

$$A = \begin{bmatrix} 5 & 2 & 1 \\ 0 & -1 & 4 \\ -2 & 3 & 7 \end{bmatrix}$$

Compute the **trace** of the matrix and discuss what it tells us about the sum of eigenvalues (hints about system stability).

Unit1Math > problem.py >

```
import numpy as np

# Vectors
f = np.array([2, -3, 4]) # force
d = np.array([-1, 2, 1]) # displacement
r = np.array([0, 1, 0]) # application point

# 1. Normalization
f_norm = f / np.linalg.norm(f)
print("Normalized force vector:", f_norm)

# 2. Work performed
work = np.dot(f, d)
print("Work performed:", work)

# 3. Torque
torque = np.cross(r, f)
print("Torque:", torque)

# 4. Trace of the matrix
A = np.array([[5, 2, 1],
              [0, -1, 4],
              [-2, 3, 7]])
trace_A = np.trace(A)
print("Trace of A:", trace_A)
```



4) Trace of the matrix

$$\text{tr}(A) = 5 + (-1) + 7 = 11$$

Interpretation: The trace equals the **sum of the eigenvalues**. By itself, it **does not determine stability** (we would need the actual eigenvalues), but it indicates that the sum of the eigenvalues is positive.





```
1  import numpy as np
2  import matplotlib.pyplot as plt
3
4  # Define the vectors a, b, and their sum c
5  a = np.array([4, -2, 1]) # Vector a
6  b = np.array([1, -1, 3]) # Vector b
7  c = a + b               # Vector c as the sum of a and b
8
9  # Define the starting points of the vectors (all starting at the origin)
10 starts = np.zeros((3, 3)) # A 3x3 array of zeros representing the origin [0, 0, 0] for all vectors
11
12 # Define the endpoints of the vectors
13 ends = np.array([a, b, c]) # Each row corresponds to the endpoint of a, b, and c
14
15 # Create a 3D plot
16 fig = plt.figure() # Initialize a new figure for the plot
17 ax = fig.add_subplot(111, projection='3d') # Add a 3D subplot to the figure
18
19 # Plot the vectors using quiver
20 ax.quiver(
21     starts[:, 0], starts[:, 1], starts[:, 2], # Starting points of the vectors (all at the origin)
22     ends[:, 0], ends[:, 1], ends[:, 2],      # Endpoints of the vectors (defined by vectors a, b, and c)
23     color=['r', 'g', 'b'],                  # Colors for each vector (red, green, blue)
24     arrow_length_ratio=0.1                  # Adjust the arrow size relative to the vector length
25 )
26
27 # Set axis labels for better understanding of the plot
28 ax.set_xlabel('X-axis')
29 ax.set_ylabel('Y-axis')
30 ax.set_zlabel('Z-axis')
31
32 # Set the aspect ratio to be equal for better visualization
33 ax.set_box_aspect([1, 1, 1]) # Make the axes scale equally
34
35 # Show the plot
36 plt.show()
37
```

Orthogonal and Orthonormal Vectors

Orthogonal Vectors

- Two vectors $\mathbf{u}$ and $\mathbf{v}$ are orthogonal if their dot product is zero:

$$\mathbf{u} \cdot \mathbf{v} = 0$$

- In three-space, multiple vectors can be mutually perpendicular.
-

Orthonormal Vectors

- Vectors that are both **orthogonal** and **unit vectors** are called orthonormal.
 - Example: If $\|\mathbf{u}\| = 1$, $\|\mathbf{v}\| = 1$, and $\mathbf{u} \cdot \mathbf{v} = 0$, then $\mathbf{u}$ and $\mathbf{v}$ are orthonormal.
-

Orthogonal Matrix

- A square matrix Q is orthogonal if:

$$Q^T Q = Q Q^T = I$$

where I is the identity matrix.

- Properties:**

- The transpose of an orthogonal matrix equals its inverse:

$$Q^T = Q^{-1}$$

Examples of Orthogonal Matrices

- Identity Transformation:

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- Rotation Matrix (e.g., rotation by 16.26°):

$$R(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$$

For $\theta = 16.26^\circ$:

$$R = \begin{bmatrix} 0.96 & -0.28 \\ 0.28 & 0.96 \end{bmatrix}$$

- Reflection Across the x-axis:

$$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

- Permutation of Coordinate Axes:

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Differentiation of Vectors

1. Definition

Suppose the vector $x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{bmatrix}$ is a function of time. Then, the time derivative $\dot{x}$ of x is

defined as:

$$\dot{x} = \begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \\ \vdots \\ \dot{x}_n(t) \end{bmatrix}.$$

2. Coordinate-Wise Differentiation

- The derivative $\frac{dA}{dt}$ of a matrix $A = (a_{ij})$ is just the matrix $\dot{A} = (\dot{a}_{ij})$.
- Similar statements hold for integration of vectors and matrices.

3. Product Rules

Scalar and vector products satisfy product rules similar to differentiation of ordinary functions:

- **Dot Product Rule:**

$$\frac{d}{dt} \langle x, y \rangle = \left\langle \frac{dx}{dt}, y \right\rangle + \left\langle x, \frac{dy}{dt} \right\rangle.$$

- **Cross Product Rule:**

$$\frac{d}{dt} (x \times y) = \frac{dx}{dt} \times y + x \times \frac{dy}{dt}.$$

This provides the rules for differentiating scalar and vector products, ensuring consistency in their application.

```
import sympy as sp # Import SymPy for symbolic computation

# Define the symbolic variable
x = sp.Symbol('x')

# Define the vector as a column matrix
X = sp.Matrix([
    sp.sin(x),      # First component: sin(x)
    x**2,           # Second component: x^2
    sp.cos(x) - 3*x # Third component: cos(x) - 3x
])

# Compute the derivative of the vector with respect to x
dX_dx = X.diff(x)

# Display the result in a readable format
sp.pprint(dX_dx)
```



symbol_derivative.py

Exercise

Problem: Matrix Reduction Using Gauss-Jordan Elimination

Given the following augmented matrix:

$$\left[\begin{array}{cccc|c} 1 & 3 & 1 & 9 & 9 \\ 1 & 1 & -1 & 1 & 1 \\ 3 & 11 & 5 & 35 & 35 \end{array}\right]$$

The goal is to transform it into its **reduced row echelon form (RREF)** using the **Gauss-Jordan elimination method**.

Step 1: Make the pivot in the first column a 1.

The pivot element in the first column is already 1 (in row 1). No operation is needed here.

Step 2: Eliminate the other elements in the first column (make them 0).

- Subtract row 1 from row 2 ($R_2 \rightarrow R_2 - R_1$):

$$R_2 = [1, 1, -1, 1] - [1, 3, 1, 9] = [0, -2, -2, -8]$$

- Subtract 3 × row 1 from row 3 ($R_3 \rightarrow R_3 - 3R_1$):

$$R_3 = [3, 11, 5, 35] - 3 \cdot [1, 3, 1, 9] = [0, 2, 2, 8]$$

Updated matrix:

$$\left[\begin{array}{cccc|c} 1 & 3 & 1 & 9 & 9 \\ 0 & -2 & -2 & -8 & -8 \\ 0 & 2 & 2 & 8 & 8 \end{array}\right]$$

Step 3: Make the pivot in the second column a 1.

- Divide row 2 by -2 ($R_2 \rightarrow R_2 / -2$):

$$R_2 = [0, -2, -2, -8] / -2 = [0, 1, 1, 4]$$

Updated matrix:

$$\left[\begin{array}{cccc|c} 1 & 3 & 1 & 9 & 9 \\ 0 & 1 & 1 & 4 & 4 \\ 0 & 2 & 2 & 8 & 8 \end{array}\right]$$

Step 4: Eliminate the other elements in the second column (make them 0).

- Subtract 3 × row 2 from row 1 ($R_1 \rightarrow R_1 - 3R_2$):

$$R_1 = [1, 3, 1, 9] - 3 \cdot [0, 1, 1, 4] = [1, 0, -2, -3]$$

- Subtract 2 × row 2 from row 3 ($R_3 \rightarrow R_3 - 2R_2$):

$$R_3 = [0, 2, 2, 8] - 2 \cdot [0, 1, 1, 4] = [0, 0, 0, 0]$$

Updated matrix:

$$\left[\begin{array}{cccc|c} 1 & 0 & -2 & -3 & -3 \\ 0 & 1 & 1 & 4 & 4 \\ 0 & 0 & 0 & 0 & 0 \end{array}\right]$$

Step 5: Make the pivot in the third column a 1 (if applicable).

In this case, the third row is all zeros, so no further operations are needed.

Final RREF Matrix:

$$\left[\begin{array}{cccc|c} 1 & 0 & -2 & -3 & -3 \\ 0 & 1 & 1 & 4 & 4 \\ 0 & 0 & 0 & 0 & 0 \end{array}\right]$$

```
from sympy import Matrix
```

```
# Define the augmented matrix
```

```
A = Matrix([
    [1, 3, 1, 9],
    [1, 1, -1, 1],
    [3, 11, 5, 35]
])
```

```
# Perform row reduction (Gauss-Jordan)
```

```
reduced_matrix = A.rref()
```

```
# Display the result
```

```
print("Reduced Row Echelon Form:")
```

```
print(reduced_matrix[0])
```

reduced_matrix = (M_rref, pivot_columns)

Linear Dependence and Independence

1. Linear Dependence

A set of vectors $\{\vec{v}_1, \vec{v}_2, \dots, \vec{v}_k\}$ in a vector space V is **linearly dependent** if there exist scalars $a_1, a_2, \dots, a_k$, **not all zero**, such that:

$$a_1\vec{v}_1 + a_2\vec{v}_2 + \dots + a_k\vec{v}_k = \vec{0}.$$

This means at least one vector in the set can be written as a combination of the others.

2. Linear Independence

A set of vectors $\{\vec{v}_1, \vec{v}_2, \dots, \vec{v}_n\}$ is **linearly independent** if the equation:

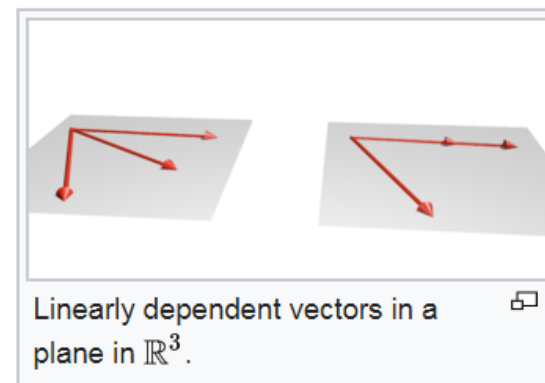
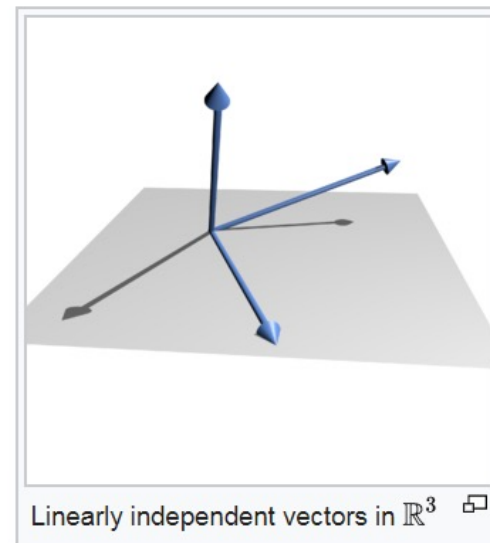
$$a_1\vec{v}_1 + a_2\vec{v}_2 + \dots + a_n\vec{v}_n = \vec{0}$$

holds **only** when **all the scalars $a_1, a_2, \dots, a_n$ are zero**.

This means none of the vectors can be expressed as a combination of the others.

Simplified Explanation:

- If you can express one vector in the set as a mix of the others, the vectors are **dependent**.
- If no vector can be made using the others, they are **independent**.



Exercise

Determine whether the set of vectors $\{v_1, v_2, v_3\}$ is linearly independent or dependent over the field of real numbers ($\mathbb{R}$) in each case.

Part (a)

$$v_1 = \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 1 \\ 1 \\ -2 \end{bmatrix}, \quad v_3 = \begin{bmatrix} -2 \\ 3 \\ 1 \end{bmatrix}$$

Part (b)

$$v_1 = \begin{bmatrix} 1 \\ 3 \\ 3 \end{bmatrix}, \quad v_2 = \begin{bmatrix} -1 \\ 1 \\ 3 \end{bmatrix}, \quad v_3 = \begin{bmatrix} -5 \\ -7 \\ 3 \end{bmatrix}$$

Part (a): Linear Independence or Dependence

To check if the vectors are linearly independent, solve the equation:

$$c_1v_1 + c_2v_2 + c_3v_3 = 0$$

This translates to solving the following augmented system:

$$\begin{bmatrix} 1 & 1 & -2 & 0 \\ -1 & 1 & 3 & 0 \\ 2 & -2 & 1 & 0 \end{bmatrix}$$

Part (b): Linear Independence or Dependence

Similarly, solve the equation:

$$c_1v_1 + c_2v_2 + c_3v_3 = 0$$

This translates to solving:

$$\begin{bmatrix} 1 & -1 & -5 & 0 \\ 3 & 1 & -7 & 0 \\ 3 & 3 & 3 & 0 \end{bmatrix}$$



linear_independence_check.py

```
1  from sympy import Matrix
2
3  # Part (a): Define the matrix with vectors as columns
4  A = Matrix([
5      [1, 1, -2],
6      [-1, 1, 3],
7      [2, -2, 1]
8  ])
9
10 # Compute the rank of the matrix
11 rank_a = A.rank()
12
13 # Check linear independence for Part (a)
14 if rank_a == A.shape[1]:
15     result_a = "Linearly Independent"
16 else:
17     result_a = "Linearly Dependent"
18
19 # Part (b): Define the matrix with vectors as columns
20 B = Matrix([
21     [1, -1, -5],
22     [3, 1, -7],
23     [3, 3, 3]
24 ])
25
26 # Compute the rank of the matrix
27 rank_b = B.rank()
28
29 # Check linear independence for Part (b)
30 if rank_b == B.shape[1]:
31     result_b = "Linearly Independent"
32 else:
33     result_b = "Linearly Dependent"
34
35 # Print results
36 print("Part (a):", result_a)
37 print("Part (b):", result_b)
38
```

Transpose of a Matrix

Transpose of a Matrix

- **Definition:**

The transpose of a matrix A , denoted as A^T , is a new matrix formed by interchanging the rows and columns of A .

- **Notation:**

If $A = [a_{ij}]$, then $A^T = [a_{ji}]$.

- **Properties:**

1. $(A^T)^T = A$
2. $(A + B)^T = A^T + B^T$
3. $(cA)^T = cA^T$ (for scalar c)
4. $(AB)^T = B^T A^T$

- **Example:**

If

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix},$$

then

$$A^T = \begin{bmatrix} 1 & 3 & 5 \\ 2 & 4 & 6 \end{bmatrix}.$$

```
import numpy as np

# Define the matrix
A = np.array([
    [1, 2],
    [3, 4],
    [5, 6]
])

# Compute the transpose
A_transpose = A.T

# Display the original matrix and its transpose
print("Original Matrix A:")
print(A)

print("\nTranspose of Matrix A (A^T):")
print(A_transpose)
```

Unit Vector

- **Definition:**

A unit vector is a vector with **magnitude equal to 1**, also referred to as a direction vector.

- **Formula:**

The unit vector $\hat{\mathbf{v}}$ in the direction of a given nonzero vector $\mathbf{v}$ is computed as:

$$\hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$$

where $\|\mathbf{v}\|$ is the norm (magnitude) of $\mathbf{v}$.

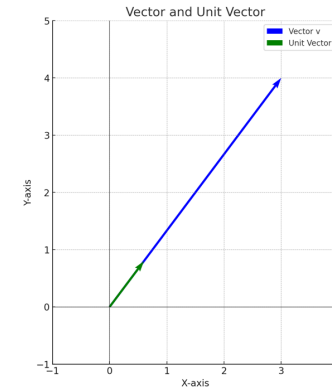
- **Properties:**

- Unit vectors retain the **direction** of $\mathbf{v}$.
- Any vector can be converted to a unit vector by dividing it by its norm.

- **Example:**

For $\mathbf{v} = [3, 4]$,

$$\|\mathbf{v}\| = \sqrt{3^2 + 4^2} = 5, \quad \hat{\mathbf{v}} = \frac{[3, 4]}{5} = [0.6, 0.8].$$



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Define the vector v
5 v = np.array([3, 4])
6
7 # Compute the magnitude (norm) of v
8 norm_v = np.linalg.norm(v)
9
10 # Compute the unit vector of v
11 unit_v = v / norm_v
12
13 # Create a 2D plot
14 plt.figure()
15 plt.quiver(0, 0, v[0], v[1], angles='xy', scale_units='xy', scale=1, color='blue', label='Vector v')
16 plt.quiver(0, 0, unit_v[0], unit_v[1], angles='xy', scale_units='xy', scale=1, color='green', label='Unit Vector \hat{v}')
17
18 # Set axis limits
19 plt.xlim(0, 4)
20 plt.ylim(0, 4.5)
21
22 # Add grid, labels, and legend
23 plt.grid()
24 plt.axhline(0, color='black', linewidth=0.5)
25 plt.axvline(0, color='black', linewidth=0.5)
26 plt.xlabel('x')
27 plt.ylabel('y')
28 plt.legend()
29 plt.title('Vector and Unit Vector')
30
31 # Show the plot
32 plt.show()
33
```

Matrix Rank

- **Definition:**

The rank of a matrix A is the maximum number of linearly independent rows or columns in A .

- **Key Points:**

- Rank represents the dimension of the column space or row space of the matrix.
- A matrix is **full rank** if its rank equals the smaller of its dimensions ($\min(n, m)$ for an $n \times m$ matrix).

- **Properties:**

- $\text{rank}(A) \leq \min(n, m)$.
- Rank remains unchanged under row or column operations.
- If A is square and full rank, $\det(A) \neq 0$.

- **Example:**

For $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$,

$\text{rank}(A) = 2$, as only 2 rows/columns are linearly independent.

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Two **columns** are linearly independent

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

Two **rows** are linearly independent

Linear Algebraic Equations

Definition:

Given the equation:

$$A\mathbf{x} = \mathbf{y},$$

where:

- A : an $m \times n$ real matrix,
- $\mathbf{x}$: an $n \times 1$ unknown vector,
- $\mathbf{y}$: an $m \times 1$ vector of constants.

The goal is to solve for $\mathbf{x}$, where:

- m : the number of equations,
- n : the number of unknowns.

Key Concepts:

1. Range Space of A :

- The range space (or column space) of A consists of all possible linear combinations of the columns of A .

2. Rank of A :

- Defined as the dimension of the range space (number of linearly independent columns of A).

3. Null Vector and Null Space:

- A vector $\mathbf{x}$ is a null vector if $A\mathbf{x} = \mathbf{0}$.
- The null space consists of all null vectors of A .

4. Nullity of A :

- The nullity is the dimension of the null space, i.e., the number of linearly independent null vectors of A .
- Related to the rank by:

$$\text{Nullity}(A) = \text{Number of Columns of } A - \text{Rank}(A).$$

Exercise

Consider the matrix:

$$A = \begin{bmatrix} 0 & 1 & 1 & 2 \\ 1 & 2 & 3 & 4 \\ 2 & 0 & 2 & 0 \end{bmatrix}$$

1. Determine the rank of A .
2. Identify the basis of the range space (column space) of A .
3. Compute the nullity of A .
4. Find the basis of the null space of A .

Finding the Basis:

1. **Null Space Definition:** The null space of a matrix A is the set of all vectors $\mathbf{n}$ such that $A\mathbf{n} = 0$.
2. **Solve $A\mathbf{n} = 0$:**
 - Write the augmented matrix for the homogeneous system $A\mathbf{n} = 0$.
 - Perform row reduction to bring the matrix to its reduced row echelon form (RREF).
 - Express the free variables in terms of the pivot variables to get the general solution.
3. **Identify Basis Vectors:**
 - The solution vector will include parameters (free variables).
 - Each parameter corresponds to a linearly independent vector in the null space.
 - Collect these vectors to form the basis of the null space.

Solution

1. Rank of A :

- A has 2 linearly independent columns ($\mathbf{a}_1$ and $\mathbf{a}_2$).
- $\text{Rank}(A) = 2$.

2. Basis of the Range Space:

- The basis of the column space is formed by the first two columns:

$$\text{Basis of Range Space: } \{\mathbf{a}_1, \mathbf{a}_2\} = \left\{ \begin{bmatrix} 0 \\ 1 \\ 2 \end{bmatrix}, \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix} \right\}$$

3. Nullity of A :

- Using the formula:

$$\text{Nullity}(A) = \text{Number of Columns} - \text{Rank}(A) = 4 - 2 = 2$$

4. Basis of the Null Space:

- Solve $A\mathbf{n} = 0$:

$$\mathbf{n}_1 = \begin{bmatrix} 1 \\ 1 \\ -1 \\ 0 \end{bmatrix}, \quad \mathbf{n}_2 = \begin{bmatrix} 0 \\ 2 \\ 0 \\ -1 \end{bmatrix}$$

- The basis of the null space is:

$$\text{Basis of Null Space: } \{\mathbf{n}_1, \mathbf{n}_2\}$$

Matrix Rank: Bridge Sensor System

The Setup: Bridge with 5 vibration sensors monitoring structural health

S1 — S2 — S3 — S4 — S5

Each sensor measures vibration at 3 different times (car, truck, motorcycle, etc).



Scenario 1: Full Rank = 3 ✓

```
python
vibrations = [[2.1, 3.5, 1.8, 4.2, 2.9], # Car
              [4.3, 7.1, 3.6, 8.5, 5.8], # Truck
              [1.5, 2.4, 1.2, 2.8, 1.9]] # Motorcycle

rank = 3 # All 3 measurements are independent
```

Engineering Decision:

- ✓ System working properly
- ✓ All measurements provide unique information
- ✓ Can detect different vibration patterns

Scenario 2: Rank = 2 ⚠

```
python
vibrations = [[2.1, 3.5, 1.8, 7.0, 2.9], # S4 = 2×S2
              [4.3, 7.1, 3.6, 14.2, 5.8], # S4 = 2×S2
              [1.5, 2.4, 1.2, 4.8, 1.9]] # S4 = 2×S2

rank = 2 # Only 2 independent measurements
```

Engineering Decision:

- ⚠ Sensor 4 is redundant (duplicate of Sensor 2)
- ⚠ Wasting money, power, and bandwidth
- ⚠ **Action:** Remove or relocate Sensor 4

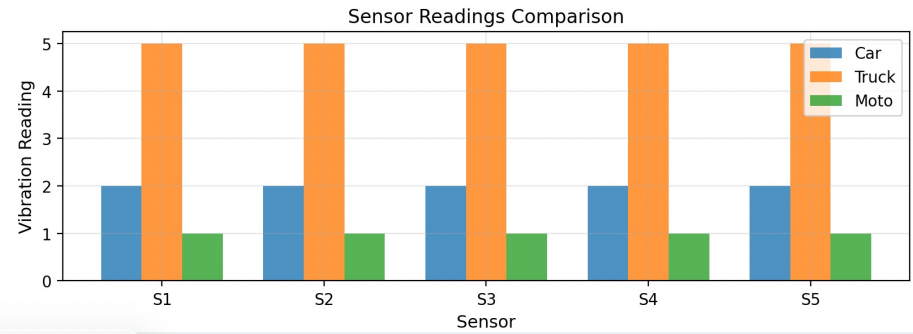
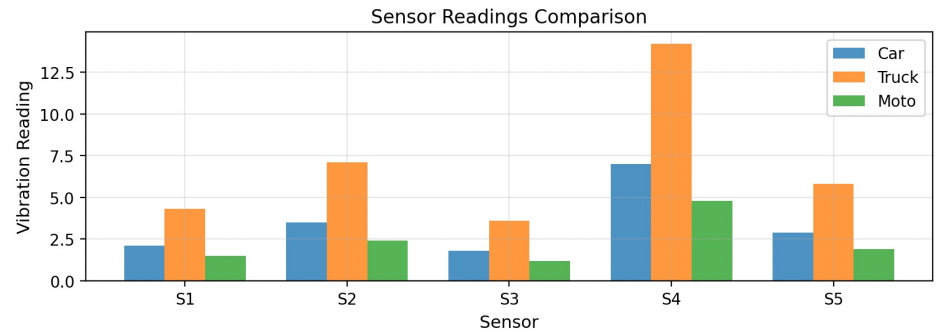
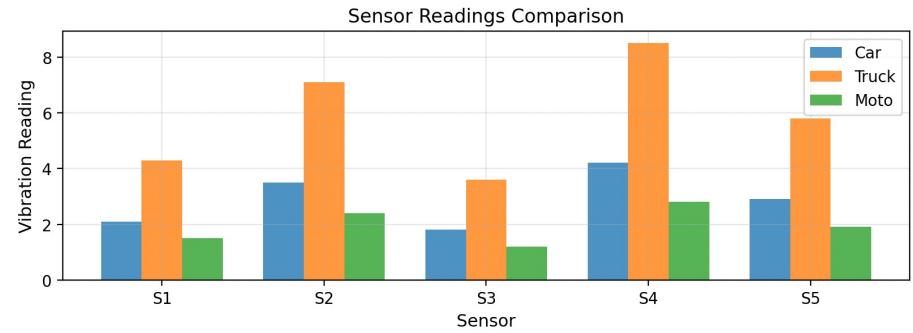
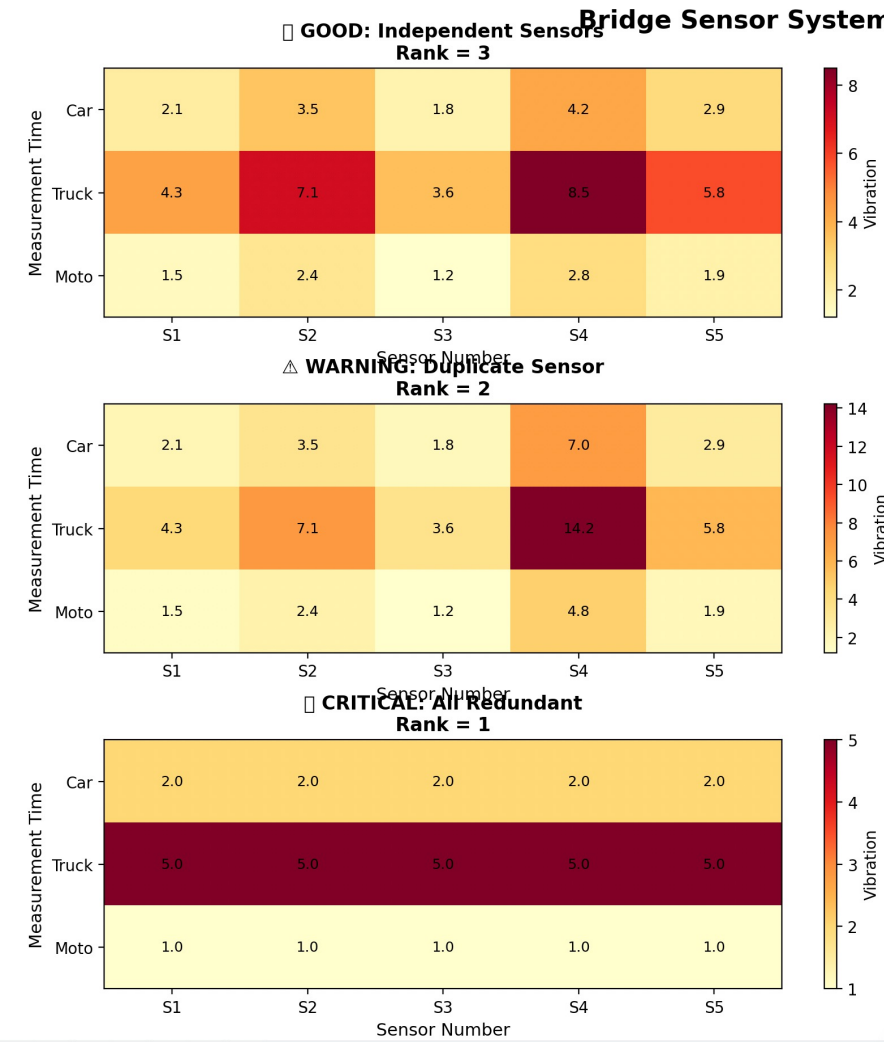
Scenario 3: Rank = 1 ✗

```
vibrations = [[2.0, 2.0, 2.0, 2.0, 2.0], # All same
              [5.0, 5.0, 5.0, 5.0, 5.0], # All same
              [1.0, 1.0, 1.0, 1.0, 1.0]] # All same

rank = 1 # All sensors reading identically
```

Engineering Decision:

- ✗ Critical system failure
- ✗ All sensors connected to same wire (installation error)
- ✗ Cannot distinguish vibration types
- ✗ **Action:** Complete system redesign required



Eigenvalues and Eigenvectors

Eigenvalues and Eigenvectors

- **Definition:**

For a square matrix A , an eigenvector $\mathbf{v}$ and eigenvalue λ satisfy:

$$A\mathbf{v} = \lambda\mathbf{v},$$

where $\mathbf{v} \neq 0$ is the eigenvector and λ is the eigenvalue.

- **Properties:**

1. Eigenvalues can be real or complex.
2. The eigenvectors corresponding to distinct eigenvalues are linearly independent.
3. The determinant of $A - \lambda I = 0$ gives the eigenvalues.

- **Applications:**

Used in stability analysis, dimensionality reduction (PCA), vibration analysis, etc.

- **Example:**

For matrix

$$A = \begin{bmatrix} 4 & 1 \\ 2 & 3 \end{bmatrix},$$

- Eigenvalues: $\lambda_1 = 5, \lambda_2 = 2$

- Eigenvectors:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \mathbf{v}_2 = \begin{bmatrix} -1 \\ 2 \end{bmatrix}.$$

```
import numpy as np

# Define the matrix
A = np.array([
    [4, 1],
    [2, 3]
])

# Compute eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)

# Display the results
print("Matrix A:")
print(A)

print("\nEigenvalues of A:")
print(eigenvalues)

print("\nEigenvectors of A (columns correspond to eigenvalues):")
print(eigenvectors)
```

1. Scaling of Eigenvectors

- Eigenvectors associated with an eigenvalue are unique only up to a **scaling factor** (multiplication by a nonzero constant).
- If $\mathbf{v}$ is an eigenvector, then any scaled version $c\mathbf{v}$ (where $c \neq 0$) is also a valid eigenvector.

Example:

If an eigenvector is $[1, 1]^T$, another valid one could be $[0.707, 0.707]^T$ (normalized) or $[2, 2]^T$ (scaled).

Eigenfaces: Computer Vision Meets Linear Algebra



eigenExample.py

The Problem:

- Each face image: 100×100 pixels = **10,000 dimensions**
- Store 1,000 faces = **10 million numbers** 🤯
- Face recognition = comparing vectors in 10,000D space (SLOW!)



Eigenfaces: Computer Vision Meets Linear Algebra

Type equation here.

The Problem

- Each face image: 100×100 pixels = **10,000 dimensions**
- Store 1,000 faces = **10 million numbers** 🤖
- Face recognition = comparing vectors in 10,000D space (SLOW!)

$$\begin{array}{l|l} x_1 = [120, 115, \dots, 95] & \leftarrow \text{face}_1 \text{ (Bush)} \\ x_2 = [110, 112, \dots, 88] & \leftarrow \text{face}_2 \text{ (Powell)} \\ x_3 = [125, 130, \dots, 102] & \leftarrow \text{face}_3 \text{ (Chávez)} \\ \vdots & \\ x_{1000} = [122, \dots, 97] & \leftarrow \text{face}_{1000} \end{array}$$

Step 1: Build the Data Matrix

$$X = \begin{bmatrix} \text{face}_1 \\ \text{face}_2 \\ \vdots \\ \text{face}_n \end{bmatrix} \quad \begin{array}{l} \text{Each row is a flattened image} \\ \text{face} \in \mathbb{R}^{10,000} \end{array}$$
$$X \in \mathbb{R}^{1,000 \times 10,000}$$

Step 2: Center the Data

$$\mu = (1/n) \sum x_i \quad (\text{mean face})$$
$$\tilde{X} = X - \mu \quad (\text{centered data})$$

Step 3: Compute Covariance Matrix

$$C = (1/n) \tilde{X}^T \tilde{X} \quad (C \text{ is } 10,000 \times 10,000)$$

Step 4: Find Eigenvectors & Eigenvalues

$$Cv = \lambda v$$

Where:

- v = eigenvector (eigenface)
- λ = eigenvalue (importance/variance captured)

Step 5: Keep Top K Eigenvectors $v_i \in \mathbb{R}^{10,000}$

Sort by λ : $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n$
Keep top K eigenfaces: $V = [v_1, v_2, \dots, v_k]$

Typically $K \approx 150$ captures 95%+ variance

Step 6: Represent Any Face

$$x \approx \mu + \sum_{i=1 \text{ to } K} w_i v_i \quad (\text{linear combination})$$

$$\text{Where: } w = V^T(x - \mu) \quad (\text{projection coefficients})$$

The Magic ✨

- **Before:** 10,000 numbers per face
- **After:** 150 numbers per face
- **Compression:** 98.5%
- **Quality:** Nearly identical!

Why It Works

The eigenvectors with largest eigenvalues capture the **most important variations** in faces:

- v_1 might encode lighting
- v_2 might encode face shape
- v_3 might encode smile vs. serious
- etc.

Real-World Impact:

- 📱 Face ID on smartphones
- 🚪 Security systems
- 📷 Photo compression
- 📱 Snapchat/Instagram filters

$$x \in \mathbb{R}^{1,000 \times 10,000}$$

x : Chavez image [approximation]

x : Chavez image [the original]

- The eigenvector IS the image.
- The eigenvalue is how important that eigenvector is.
- It is a SINGLE set of eigenvectors unique to ALL people.

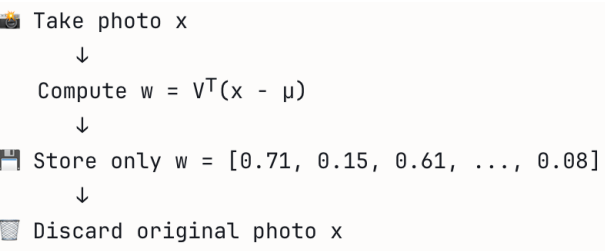
First 18 Eigenfaces (eigenvectors v_i)



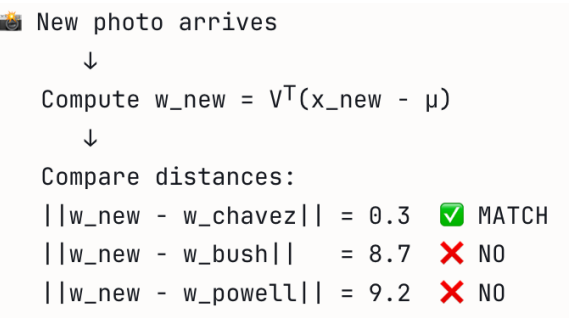
Why Eigenfaces? The Real Value is **w**
w = 150 numbers that uniquely identify a face

What Actually Happens in Practice

Step 1: Enrollment (once per person)



Step 2: Recognition (every time)



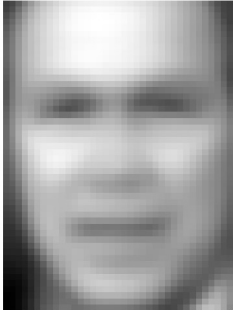
You NEVER reconstruct the image. You only compare **w** vectors.

The Real Numbers		
	Without Eigenfaces	With Eigenfaces
Store 1M faces	10 billion numbers	150 million numbers
Compare 2 faces	10,000 dimensions	150 dimensions
Speed	🐢 Slow	⚡ 66× faster
Privacy	Real photo stored	Only abstract numbers

Bottom Line
w is the numeric fingerprint of a face. No photo needed — just 150 numbers.

Reconstructing a face with different numbers of eigenfaces

5 eigenfaces



10 eigenfaces



25 eigenfaces



50 eigenfaces



100 eigenfaces



150 eigenfaces



200 eigenfaces



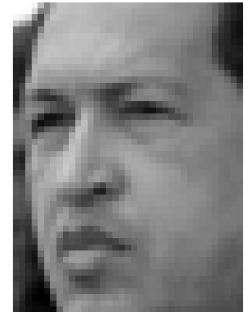
300 eigenfaces



500 eigenfaces



1850 eigenfaces



Original image



200,000 px



We extract only the face
(pre-processed)



Output from the
algorithm and
resized to 50×37

500 eigenfaces



2×2 Systems

1. Mass-spring-damper (the one you already have)

State: $x = [\text{position}, \text{velocity}]^T$

$$A = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}$$

Used in: suspension systems, seismic isolators, building dampers.

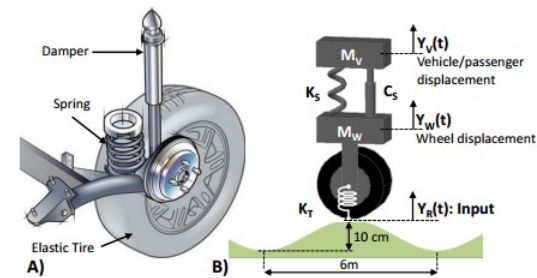


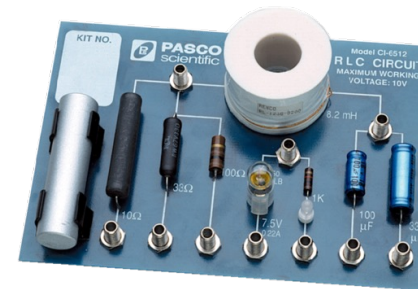
Figure 1: Suspension System

2. RLC electrical circuit

A series circuit with resistor R, inductor L, capacitor C. State: $x = [\text{voltage across C}, \text{current through L}]^T$

$$A = \begin{bmatrix} 0 & 1/C \\ -1/L & -R/L \end{bmatrix}$$

Mathematically **identical structure** to the mass-spring-damper. $k \leftrightarrow 1/C$, $c \leftrightarrow R$, $m \leftrightarrow L$. This is why electrical and mechanical engineers use the same math.



3. DC motor

State: $x = [\text{angular velocity } \omega, \text{current } i]^T$

$$A = \begin{bmatrix} -b/J & K/J \\ -K/L & -R/L \end{bmatrix}$$

Where J = inertia, b = friction, K = motor constant, R = resistance, L = inductance. Used in robotics, CNC machines, drones.



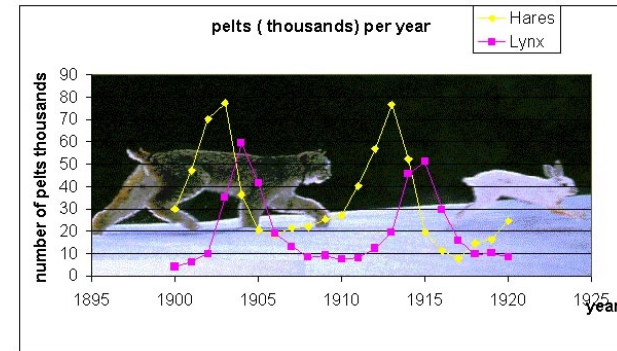
4. Predator-prey (linearized Lotka-Volterra)

Near equilibrium, the population dynamics of two species linearize to:

State: $x = [\delta \text{prey}, \delta \text{predator}]^T$ (small deviations from equilibrium)

$$A = \begin{bmatrix} 0 & -a \\ b & 0 \end{bmatrix}$$

Eigenvalues are purely imaginary $\rightarrow$ the linearized system oscillates. Used in ecology and epidemiology.



5. Two interconnected tanks (fluid level control)

Two tanks connected by a pipe. State: $x = [\text{height } h_1, \text{height } h_2]^T$

$$A = \begin{bmatrix} -a_1 & a_1 \\ a_1 & -a_1 \end{bmatrix}$$

Where a_1 depends on pipe geometry and gravity. Used in chemical plants, water treatment systems.



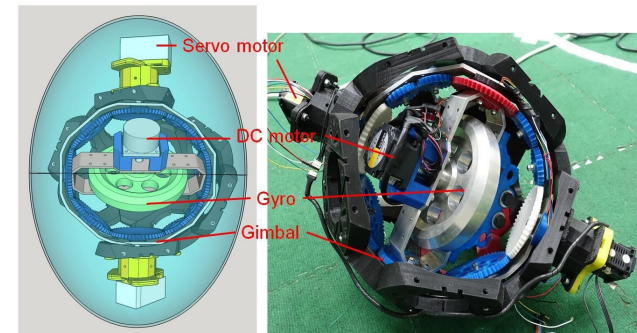
3x3 Systems

6. Three-phase DC motor / gyroscope

State: $x = [\omega_x, \omega_y, \omega_z]^T$ (angular velocities about three axes)

$$A = \begin{bmatrix} 0 & \omega_z & -\omega_y \\ -\omega_z & 0 & \omega_x \\ \omega_y & -\omega_x & 0 \end{bmatrix}$$

This is a **skew-symmetric** matrix $\rightarrow$ eigenvalues are purely imaginary $\rightarrow$ the motion is purely rotational with no energy loss. Used in spacecraft attitude control and gyroscopes.



Exercise: Stability Analysis of a Linear System with Varying Dynamics

We aim to analyze the behavior of a linear system of first-order differential equations under different dynamic conditions, characterized by different choices of the system matrix A . The system is given by:

$$\dot{\mathbf{x}} = A\mathbf{x}$$

where:

$$A \in \mathbb{R}^{2 \times 2}, \quad \mathbf{x}(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix}.$$

Instructions



eigSystem.py

1. **Define the system function:** Implement the differential equation in a Python function.
 2. **Experiment with different system matrices:** Choose different matrices A that yield:
 - Real and distinct eigenvalues.
 - Complex conjugate eigenvalues.
 - Repeated eigenvalues.
 - Eigenvalues with positive, negative, or zero real parts.
 3. **Solve the system:** Use `solve_ivp` to integrate the differential equation over the time interval $t \in [0, 10]$.
 4. **Explore different initial conditions:** Try multiple initial conditions and analyze how the system evolves in each case.
 5. **Plot the solutions:** For each combination of A and initial conditions, plot $x_1(t)$ and $x_2(t)$
-

🔥 Bonus Challenge! 🔥

Students who complete the exercises by **Wednesday the 18th** will receive **one extra point** on the midterm exam.

Please submit a **printed sheet with your answers** at the end of class on that day. Email and PDF submissions will not be accepted.

 **17 Reminder:** There will be **no class next Monday the 16th**, so take advantage of this extra time to work on the exercises and secure your bonus point.

Example: If you score **8/10**, your final grade will become **9/10**.

Exercises

- 1. Eigenvalue classification** Given the matrix $A = \begin{bmatrix} -1 & 2 \\ -2 & -1 \end{bmatrix}$, compute the eigenvalues by hand. Are they real, complex, or repeated? What does the sign of their real part tell you about the long-term behavior of $x(t)$?
- 2. Stability from the trace and determinant** Without computing eigenvalues explicitly, a 2×2 system $\dot{x} = Ax$ is stable if $\text{tr}(A) < 0$ and $\det(A) > 0$. Verify this rule on at least two of your matrices.
- 3. Effect of initial conditions** For a fixed matrix A with complex conjugate eigenvalues, run the system with three different initial conditions $x(0)$. Do the trajectories converge to the same point? Do they follow the same path? Explain why the initial condition scales the solution but does not change its qualitative shape.
- 4. The repeated eigenvalue case.** Consider these two matrices, both with eigenvalue $\lambda = -2$ (multiplicity 2): $A_1 = \begin{bmatrix} -2 & 0 \\ 0 & -2 \end{bmatrix}$ and $A_2 = \begin{bmatrix} -2 & 1 \\ 0 & -2 \end{bmatrix}$, where A_1 is a scalar multiple of the identity (fully diagonalizable). A_2 has a Jordan block (**not** diagonalizable). Using $x(0) = [1, 0]^T$, simulate both systems and answer:
 - Both matrices share the exact same eigenvalues. Does the simulation look the same for both? Why or why not?
 - Both systems are stable since $\text{Re}(\lambda) < 0$. But is their transient response identical? What does this tell you about the limits of using eigenvalues alone to characterize a system's behavior?

BONUS QUESTION:

- 5. From 2nd-order ODE to state space** The mass-spring-damper equation $m\ddot{x} + c\dot{x} + kx = 0$ is a second-order ODE. Rewrite it as $\dot{x} = Ax$ with $x = [x, \dot{x}]^T$ and identify the entries of A in terms of m, c, k . For what values of c does the system become unstable? Verify your answer by plugging those values into the simulation and observing the eigenvalues.

1 Stable (Both Eigenvalues Have Negative Real Parts)

Condition: $\text{Re}(\lambda_1) < 0, \text{Re}(\lambda_2) < 0$

Solution: The system converges to equilibrium at $\mathbf{x} = 0$ with exponential decay.

✓ Example Matrix A :

$$A = \begin{bmatrix} -2 & 1 \\ -1 & -3 \end{bmatrix}$$

◆ Eigenvalues: $\lambda_1 = -1, \lambda_2 = -4$ (both negative).

2 Unstable (At Least One Eigenvalue Has a Positive Real Part)

Condition: $\text{Re}(\lambda_1) > 0$ or $\text{Re}(\lambda_2) > 0$

Solution: The system exhibits exponential growth.

✓ Example Matrix A :

$$A = \begin{bmatrix} 2 & 1 \\ 0 & 1 \end{bmatrix}$$

◆ Eigenvalues: $\lambda_1 = 2, \lambda_2 = 1$ (both positive).

3 Oscillatory Stable (Complex Eigenvalues with Negative Real Part)

Condition: $\lambda = \alpha \pm j\beta$ with $\alpha < 0$

Solution: Oscillatory motion with damping.

✓ Example Matrix A :

$$A = \begin{bmatrix} -1 & -2 \\ 2 & -1 \end{bmatrix}$$

◆ Eigenvalues: $\lambda_1, \lambda_2 = -1 \pm j2$ (negative real part, damped oscillations).

4 Oscillatory Unstable (Complex Eigenvalues with Positive Real Part)

Condition: $\lambda = \alpha \pm j\beta$ with $\alpha > 0$

Solution: Oscillatory motion with exponential growth.

✓ Example Matrix A :

$$A = \begin{bmatrix} 1 & -2 \\ 2 & 1 \end{bmatrix}$$

◆ Eigenvalues: $\lambda_1, \lambda_2 = 1 \pm j2$ (positive real part, growing oscillations).

5 Neutrally Stable (Purely Imaginary Eigenvalues)

Condition: $\lambda = \pm j\beta$

Solution: Sustained oscillations without damping.

✓ Example Matrix A :

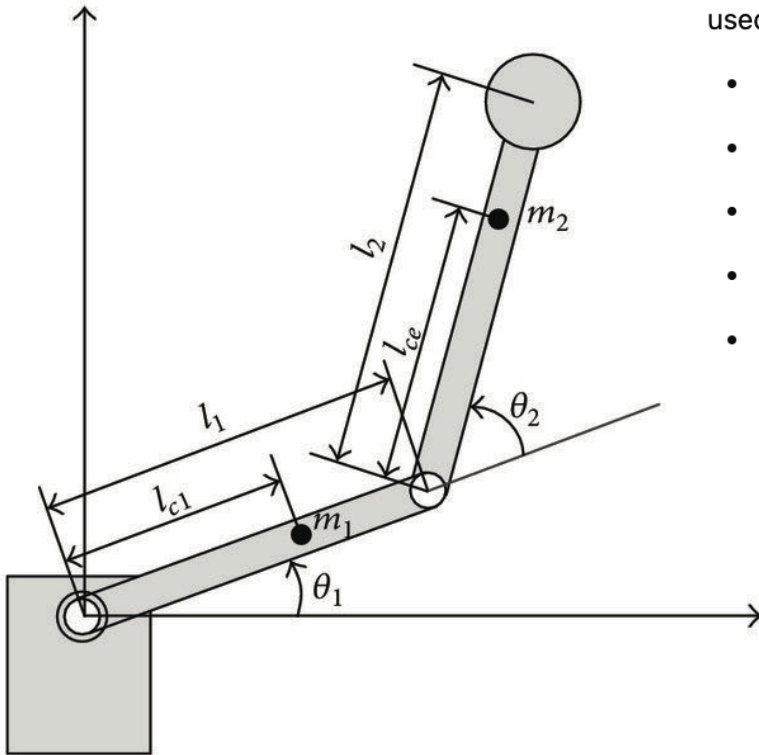
$$A = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$$

◆ Eigenvalues: $\lambda_1, \lambda_2 = \pm j1$ (pure oscillations, no decay or growth).

Conclusion

- **Negative real part → Stable solution** (system decays to zero).
- **Positive real part → Unstable solution** (system grows exponentially).
- **Imaginary component → Oscillatory behavior** (with or without damping).

Two-Link Planar Manipulator



The figure illustrates a **two-link planar robotic manipulator**, a common model in robotics used for kinematic and dynamic analysis. The notation follows standard conventions:

- θ_1, θ_2 : Joint angles that define the position of each link relative to the previous one.
- l_1, l_2 : Lengths of the first and second links, respectively.
- l_{c1}, l_{c2} : Distances from each joint to the center of mass of the respective links.
- m_1, m_2 : Masses of the links, with their centers of mass indicated by the black dots.
- **End-effector (circular shape)**: The terminal point of the second link, representing where a tool or sensor would be attached.

Exercise: Singularity Analysis of a 2-DOF Robotic Manipulator

Problem Statement

A 2-DOF planar robotic manipulator consists of two links with lengths L_1 and L_2 , connected by two revolute joints. The end-effector position is determined by the joint angles θ_1 and θ_2 .

In this exercise, you will analyze the singularity conditions of this manipulator by evaluating its Jacobian matrix.

Given Jacobian Matrix

The Jacobian matrix $J(\theta_1, \theta_2)$ for the manipulator's end-effector velocity is:

$$J(\theta_1, \theta_2) = \begin{bmatrix} -L_1 \sin \theta_1 - L_2 \sin (\theta_1 + \theta_2) & -L_2 \sin (\theta_1 + \theta_2) \\ L_1 \cos \theta_1 + L_2 \cos (\theta_1 + \theta_2) & L_2 \cos (\theta_1 + \theta_2) \end{bmatrix}$$

Tasks

1. Compute the determinant of J and determine when $\det(J) = 0$.
2. Find the singular configurations, i.e., values of θ_1 and θ_2 where the manipulator loses rank.
3. Explain the effect of singularities on the robot's motion. What happens when the determinant is zero?
4. Visualize the singular configurations by plotting them over a range of joint values.

 **Hint:** A singularity occurs when the determinant of the Jacobian is zero, meaning the robot loses the ability to move in certain directions.

What is the Jacobian Matrix?

The **Jacobian matrix** in robotics describes how the motion of a robot's **joint variables** (e.g., angles of revolute joints) translates into the **velocity of the end-effector** in Cartesian space.

For a **robotic manipulator**, the Jacobian J is a matrix that relates the joint velocities $\dot{\theta}$ to the **linear** and **angular velocity** of the end-effector:

$$\dot{\mathbf{x}} = J(\theta)\dot{\theta}$$

where:

- $\dot{\mathbf{x}}$ represents the end-effector velocity,
- $J(\theta)$ is the Jacobian matrix, which depends on the current joint angles,
- $\dot{\theta}$ represents the joint velocities.

Why is the Jacobian Important?

- It determines how joint movements affect the robot's position and orientation.
- It helps in **inverse kinematics**, where we compute the required joint velocities to achieve a desired motion.
- It is crucial for **singularity analysis**, since when $\det(J) = 0$, the robot loses mobility in some directions, making it harder or impossible to move in certain ways.

 In this exercise, we will analyze when the Jacobian becomes singular and what that means for the manipulator's motion.